🚀 Collections Framework Masterclass

Chapter 13 — PriorityQueue (Complete Deep Dive)

> **Interview Importance:** ⭐⭐⭐⭐⭐

`PriorityQueue` is one of the most important Queue implementations for interviews because it introduces a completely different concept:

> **The head is determined by priority, not insertion order.**

Internally, `PriorityQueue` uses a **binary heap**, specifically a **min-heap by default**.

---

1. What is PriorityQueue?

Package:

```
java.util.PriorityQueue
```

Declaration:

```
public class PriorityQueue<E>
    extends AbstractQueue<E>
    implements Serializable
```

Key characteristics:

- Allows duplicate elements
- Does not allow `null`
- Not thread-safe
- Default ordering is natural ordering
- Custom ordering can be supplied using `Comparator`
- Head contains the highest-priority element
- Default priority = smallest element
- Internally uses a binary heap

---

2. PriorityQueue Is Not FIFO

This is the first thing to remember.

Normal Queue:

```
10 → 20 → 30

poll() → 10
```

PriorityQueue:

```
Queue<Integer> queue =
    new PriorityQueue<>();

queue.offer(30);
queue.offer(10);
queue.offer(20);
```

The head is:

```
10
```

because `10` has the highest priority under natural ordering.

So:

```
Insertion order ≠ Processing order
```

---

3. Default Priority

For numbers:

```
Smallest number
      ↓
Highest priority
```

Example:

```
PriorityQueue<Integer> pq =
    new PriorityQueue<>();

pq.offer(50);
pq.offer(10);
pq.offer(30);
pq.offer(20);
```

Repeated `poll()`:

```
10
20
30
50
```

This behaves like a **min-priority queue**.

---

4. Internal Structure — Binary Heap

PriorityQueue is backed by an array representing a **complete binary tree**.

Conceptually:

```
            10
           /
         20    30
        /
      50    40
```

The heap property for a min-heap is:

```
Parent <= Children
```

Therefore the smallest element is always at the root.

---

5. Array Representation

The tree:

```
            10
           /
         20    30
        /
      50    40
```

is represented in an array approximately as:

```
Index:  0   1   2   3   4

Value: 10  20  30  50  40
```

This is why PriorityQueue doesn't need separate Node objects like LinkedList.

---

6. Why Complete Binary Tree?

A heap must be a **complete binary tree**.

That means:

- Every level is completely filled except possibly the last.
- The last level is filled from left to right.

Example:

```
      10
     /
   20    30
  /
 40   50
```

Valid heap structure.

This makes the tree compact and efficiently representable using an array.

---

7. Parent/Child Index Formula

For an element at index `i`:

Parent

```
(i - 1) / 2
```

Left Child

```
2 * i + 1
```

Right Child

```
2 * i + 2
```

Example:

```
Index 0
   ↓
Left  = 1
Right = 2
```

For index `1`:

```
Parent = 0
Left   = 3
Right  = 4
```

These formulas are frequently asked in coding interviews.

---

8. What Happens During `offer()`?

Suppose:

```
pq.offer(5);
```

Conceptually:

```
New element
    ↓
```

```
Place at end of heap
    ↓
Compare with parent
    ↓
Parent larger?
    ↓
Swap
    ↓
Continue upward
```

This is called:

Sift Up

or

Bubble Up

---

9. Sift-Up Example

Suppose the heap is:

```
        10
       /
     20    30
```

Insert:

```
5
```

Initially:

```
        10
       /
     20    30
    /
   5
```

`5 < 20`

Swap:

```
        10
       /
      5    30
     /
    20
```

Then:

```
5 < 10
```

Swap again:

```
        5
       /
     10   30
    /
   20
```

Heap property restored.

---

10. Complexity of `offer()`

The new element may move from the bottom to the root.

Heap height:

```
O(log n)
```

Therefore:

```
offer() → O(log n)
```

---

11. What Happens During `peek()`?

```
pq.peek();
```

The smallest element is always at the root.

Array index:

```
0
```

Therefore:

```
peek() → O(1)
```

No restructuring is necessary.

---

12. What Happens During `poll()`?

This is one of the most important PriorityQueue questions.

Suppose:

```
       10
      /
    20    30
   /
  40   50
```

`poll()` removes:

```
10
```

The last element:

```
50
```

is moved to the root.

```
      50
     /
   20    30
  /
 40
```

Now heap property is violated.

Java performs:

Sift Down

---

13. Sift-Down Example

Current:

```
      50
     /
   20    30
```

Smallest child is `20`.

Swap:

```
      20
     /
   50    30
```

Still:

```
50 > 30
```

Swap again if necessary.

Eventually:

```
      20
     /
   40    30
```

Heap property is restored.

Therefore:

```
poll() → O(log n)
```

---

14. Complexity Table

Operation	Complexity
`peek()`	O(1)
`offer()`	O(log n)
`add()`	O(log n)
`poll()`	O(log n)
`remove()`	O(log n) for head removal
`contains()`	O(n)
`remove(Object)`	O(n)
`size()`	O(1)

The key operations are:

```
peek → O(1)

offer → O(log n)

poll → O(log n)
```

---

15. Important Trap — Iterating PriorityQueue

Consider:

```
PriorityQueue<Integer> pq =
    new PriorityQueue<>();

pq.offer(30);
pq.offer(10);
pq.offer(20);
pq.offer(5);
```

Don't assume:

```
for (Integer x : pq) {
    System.out.println(x);
}
```

will print:

```
5
10
20
30
```

It is **not guaranteed to iterate in sorted order**.

The heap guarantees the correct **head**, not sorted iteration.

---

16. How to Get Elements in Priority Order?

Use repeated `poll()`:

```
while (!pq.isEmpty()) {
    System.out.println(pq.poll());
}
```

Output:

```
5
10
```

```
20
30
```

Important distinction:

```
peek/poll → priority order
iterator  → no sorted-order guarantee
```

---

17. PriorityQueue vs Sorted Collection

Don't confuse:

```
PriorityQueue
```

with:

```
TreeSet
```

PriorityQueue guarantees the highest-priority element at the head.

It does **not** maintain all elements in globally sorted iteration order.

TreeSet maintains sorted ordering of its elements.

---

18. PriorityQueue vs TreeSet

Feature	PriorityQueue	TreeSet
Duplicates	Allowed	Not allowed
Ordering	Head priority	Entire set sorted
Internal structure	Binary heap	Red-Black Tree
`peek()`	O(1)	O(log n) typically
Insert	O(log n)	O(log n)
Remove head/min	O(log n)	O(log n)
Random access	No	No

Feature	PriorityQueue	TreeSet
Null	No	Generally no

Use PriorityQueue when you mainly care about repeatedly processing the **next highest-priority item**.

---

19. Custom Priority

Default:

```
Smallest → highest priority
```

But you can reverse it.

```
PriorityQueue<Integer> pq =
    new PriorityQueue<>(
        Comparator.reverseOrder()
    );
```

Now:

```
50
40
30
20
10
```

The largest number has the highest priority.

This effectively creates a **max-priority queue**.

---

20. Max-Heap Example

```
PriorityQueue<Integer> maxHeap =
    new PriorityQueue<>(
        Comparator.reverseOrder()
    );

maxHeap.offer(10);
```

```
maxHeap.offer(50);
maxHeap.offer(20);

System.out.println(maxHeap.poll());
```

Output:

```
50
```

---

21. Custom Objects

Suppose:

```
class Task {

    int priority;
    String name;

    Task(int priority, String name) {
        this.priority = priority;
        this.name = name;
    }
}
```

Create:

```
PriorityQueue<Task> tasks =
    new PriorityQueue<>(
        Comparator.comparingInt(
            t -> t.priority
        )
    );
```

Now:

```
tasks.offer(
    new Task(3, "Email")
);

tasks.offer(
```

```
    new Task(1, "Payment")
);

tasks.offer(
    new Task(2, "Report")
);
```

Processing order:

```
Payment
Report
Email
```

assuming smaller priority number means higher priority.

---

22. Multiple Sorting Conditions

Very common interview requirement.

Suppose:

```
Priority
+
If same priority → timestamp
```

Use:

```
PriorityQueue<Task> queue =
    new PriorityQueue<>(
        Comparator
            .comparingInt((Task t) -> t.priority)
            .thenComparing(t -> t.timestamp)
    );
```

This allows sophisticated scheduling policies.

---

23. PriorityQueue and Duplicates

Unlike Set:

```
PriorityQueue allows duplicates
```

Example:

```
pq.offer(10);
pq.offer(10);
pq.offer(10);
```

All three elements remain.

```
10
10
10
```

This is an important difference from TreeSet.

---

24. Null Elements

PriorityQueue does not permit `null`.

```
pq.offer(null);
```

throws:

```
NullPointerException
```

Why?

The Queue needs to compare elements according to natural ordering or the supplied Comparator.

`null` does not have a natural ordering by itself.

---

25. Initial Capacity

You can provide an initial capacity:

```
PriorityQueue<Integer> pq =
    new PriorityQueue<>(100);
```

This controls the initial backing-array capacity.

It does **not** mean:

```
100 elements are inserted
```

It's simply storage capacity.

---

26. Does PriorityQueue Grow?

Yes.

It uses a dynamically growing array.

When the backing array becomes full:

```
Current array
      ↓
Grow
      ↓
Copy elements
      ↓
Continue
```

The exact growth strategy is implementation-specific.

Don't apply ArrayList's growth formula here.

---

27. PriorityQueue Is Not Thread-Safe

Important:

```
PriorityQueue<Integer> pq =
    new PriorityQueue<>();
```

is not thread-safe.

If multiple threads need concurrent priority-queue behavior, Java provides:

```
PriorityBlockingQueue
```

from:

```
java.util.concurrent
```

We'll cover concurrent collections later.

---

28. PriorityBlockingQueue

For concurrent applications:

```
BlockingQueue<Task> queue =
    new PriorityBlockingQueue<>();
```

It combines:

```
Priority ordering
+
BlockingQueue semantics
```

Important nuance:

`PriorityBlockingQueue` is **unbounded** by design.

---

29. Real-World Use Cases

Task Scheduling

```
Critical task
     ↓
High priority
```

```
     ↓
Normal task
```

Hospital/Support Ticket Priority

```
Emergency
High
Medium
Low
```

A PriorityQueue can process the most important item first.

Dijkstra's Algorithm

PriorityQueue is commonly used to repeatedly select the node with the smallest tentative distance.

A* Search

PriorityQueue can select the node with the smallest estimated total cost.

Top-K Problems

Extremely important for coding interviews.

Examples:

- K largest elements
- K smallest elements
- Top K frequent elements
- K closest points

PriorityQueue is one of the standard tools for these problems.

---

30. ⭐ Top-K Example

Suppose:

```
10, 40, 20, 50, 30
```

Need:

```
Top 3 largest
```

A common strategy is to maintain a min-heap of size K.

Conceptually:

```
Heap size ≤ 3
```

Process elements one by one.

If the new element is larger than the smallest heap element:

```
remove smallest
insert new element
```

At the end, the heap contains the top 3 candidates.

Complexity:

```
O(n log k)
```

This is much better than sorting all `n` elements when `k` is small.

---

31. Why Min-Heap for Top-K Largest?

Suppose K = 3.

We want to keep:

```
50
40
30
```

The smallest among our current top 3 is:

```
30
```

If `45` arrives:

```
45 > 30
```

Remove 30 and insert 45.

Now:

```
50
45
40
```

The root always represents the weakest member of the current Top-K set.

This makes replacement efficient.

---

32. ⭐ Coding Interview Pattern

K Largest Elements

```
PriorityQueue<Integer> minHeap =
    new PriorityQueue<>();

for (int n : numbers) {

    minHeap.offer(n);

    if (minHeap.size() > k) {
        minHeap.poll();
    }
}
```

At the end:

```
minHeap
→ contains K largest elements
```

Complexity:

```
O(n log k)
```

Space:

```
O(k)
```

This is an extremely useful pattern to remember.

---

33. PriorityQueue vs ArrayDeque

Feature	PriorityQueue	ArrayDeque
Ordering	Priority	FIFO/deque order
Internal structure	Binary heap	Resizable array/deque
`peek()`	Highest priority	Front
`offer()`	O(log n)	O(1) amortized
`poll()`	O(log n)	O(1)
Duplicates	Yes	Yes
Null	No	No

Choose based on ordering requirements.

---

34. PriorityQueue vs LinkedList

Feature	PriorityQueue	LinkedList
Normal FIFO	❌	✅
Priority ordering	✅	❌
Structure	Heap	Doubly linked list
`peek()`	O(1)	O(1)
`poll()`	O(log n)	O(1)
Duplicates	Yes	Yes

Feature	PriorityQueue	LinkedList
Null	❌	✅

---

35. Common Interview Traps

Trap 1

> PriorityQueue maintains all elements in sorted order.

❌ No.

Only the head is guaranteed to have the highest priority.

Trap 2

> PriorityQueue follows FIFO.

❌ No.

It follows its priority ordering.

Trap 3

> `peek()` is O(log n).

❌ No.

The root is directly available.

```
peek() → O(1)
```

Trap 4

> `poll()` is O(1).

❌ No.

After removing the root, heap restructuring may be necessary.

```
poll() → O(log n)
```

Trap 5

> PriorityQueue doesn't allow duplicates.

❌ Wrong.

Duplicates are allowed.

Trap 6

> PriorityQueue is thread-safe.

❌ Wrong.

Use `PriorityBlockingQueue` for the relevant concurrent/blocking use case.

---

36. Frequently Asked Interview Questions

Q1. What is PriorityQueue?

A Queue implementation that processes elements according to priority rather than normal FIFO insertion order.

Q2. What data structure does PriorityQueue use internally?

A binary heap stored in an array.

Q3. What is the default heap?

Min-heap.

The smallest element has the highest priority.

Q4. What is the complexity of `peek()`?

O(1).

Q5. What is the complexity of `offer()`?

O(log n).

Q6. What is the complexity of `poll()`?

O(log n).

Q7. Does PriorityQueue allow duplicates?

Yes.

Q8. Does PriorityQueue allow null?

No.

Q9. How do you create a max-heap?

```
PriorityQueue<Integer> pq =
    new PriorityQueue<>(
        Comparator.reverseOrder()
    );
```

Q10. Does iterating over PriorityQueue produce sorted output?

No.

Use repeated `poll()` if you need elements processed in priority order.

Q11. What is a heap?

A complete binary tree satisfying a heap-order property.

Q12. Why is PriorityQueue efficient?

The heap keeps the highest-priority element at the root while maintaining O(log n) insertion and removal.

Q13. What is `PriorityBlockingQueue`?

A concurrent, blocking-capable priority queue implementation.

---

37. ⭐ Quick Interview Test

What is the output?

```
PriorityQueue<Integer> pq =
    new PriorityQueue<>();

pq.offer(40);
pq.offer(10);
pq.offer(30);
pq.offer(20);

System.out.println(pq.peek());
System.out.println(pq.poll());
System.out.println(pq.poll());
```

Answer:

```
10
10
20
```

---

38. ⭐ Tricky Question

What happens here?

```
PriorityQueue<Integer> pq =
    new PriorityQueue<>();

pq.add(30);
```

```
pq.add(10);
pq.add(20);

System.out.println(pq);
```

Can you guarantee:

```
[10, 20, 30]
```

?

No.

The `toString()` /iterator view does not guarantee sorted order.

If you need priority order:

```
while (!pq.isEmpty()) {
    System.out.println(pq.poll());
}
```

---

39. ⭐ Important Coding Pattern

Find K Smallest Elements

Use a **max-heap of size K**.

Why?

The largest among the current K smallest elements stays at the root.

If a smaller value arrives:

```
remove largest
insert smaller
```

Complexity:

```
O(n log k)
```

This is the opposite of the Top-K Largest pattern.

---

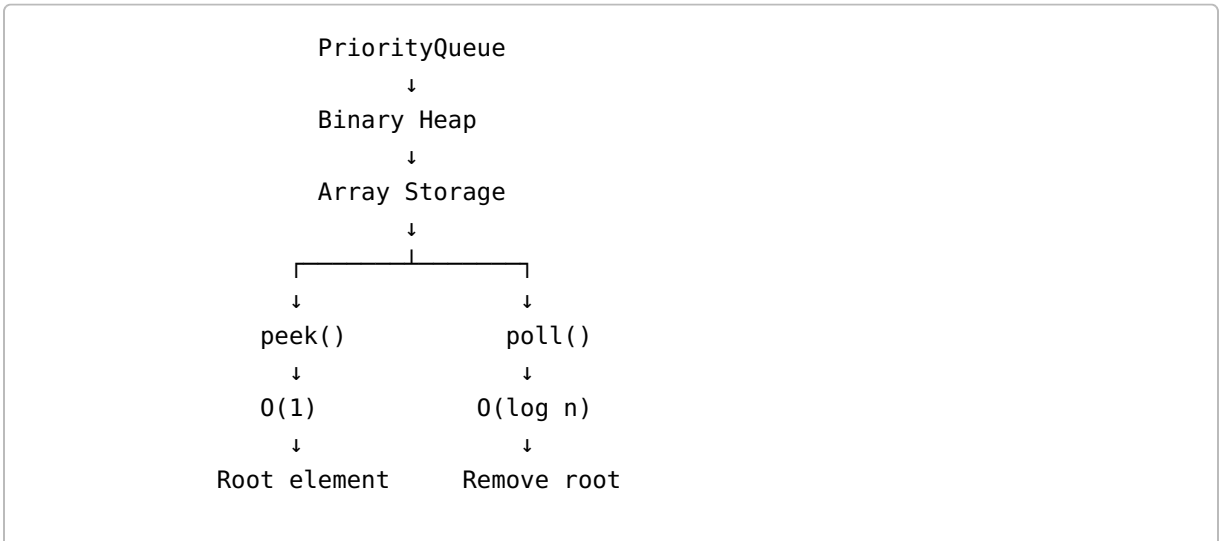
40. Decision Framework

```
Need Queue?
    ↓
Need FIFO?
   ↙       ↘
 YES       NO
 ↓          ↓
ArrayDeque  Need priority?
            ↓
      PriorityQueue
```

For concurrent priority processing:

```
Priority + concurrency
        ↓
PriorityBlockingQueue
```

🧠 Final Mental Model



```
                ↓
            Sift Down
```

Insertion:

```
New element
     ↓
End of heap
     ↓
Sift Up
     ↓
Correct heap position
```

The three operations to memorize:

```
peek()  → O(1)
offer() → O(log n)
poll()  → O(log n)
```

🔥 Chapter Summary

Concept	PriorityQueue
Type	Queue
Ordering	Priority
Default priority	Smallest element
Internal structure	Binary heap
Heap type	Min-heap by default
Storage	Resizable array
`peek()`	O(1)
`offer()`	O(log n)
`poll()`	O(log n)
Duplicates	Allowed
Null	Not allowed
Thread-safe	No

Concept	PriorityQueue
Max-heap	`Comparator.reverseOrder()`
Sorted iteration	❌
Priority processing	✅
Concurrent alternative	`PriorityBlockingQueue`

🎯 Interview Readiness Checklist

You should be able to explain:

- [ ] Why PriorityQueue is not FIFO
- [ ] What a binary heap is
- [ ] Min-heap vs max-heap
- [ ] How `offer()` works
- [ ] What sift-up means
- [ ] How `poll()` works
- [ ] What sift-down means
- [ ] Why `peek()` is O(1)
- [ ] Why `poll()` is O(log n)
- [ ] Why iteration isn't sorted
- [ ] How to create a max-heap
- [ ] How duplicates behave
- [ ] Why null isn't allowed
- [ ] PriorityQueue vs TreeSet
- [ ] PriorityQueue vs ArrayDeque
- [ ] How PriorityQueue solves Top-K problems
- [ ] What `PriorityBlockingQueue` is

🚀 Next Chapter

Chapter 14 — Map Interface (Complete Deep Dive)

We'll move from `Collection` to the other major branch of the Java Collections Framework:

```
Map
```

We'll cover:

- Why Map is not a Collection
- Key-value model
- Key uniqueness
- `put()`, `get()`, `remove()`
- `containsKey()` vs `containsValue()`
- `keySet()`, `values()`, `entrySet()`
- HashMap
- LinkedHashMap
- TreeMap
- Hashtable
- ConcurrentHashMap
- WeakHashMap
- IdentityHashMap
- EnumMap
- Map implementation selection
- The foundation for the **complete HashMap internals deep dive**